00-playwright-notes

Playwright Notes

iFrame

Iframe is two seperate html code file integrated into one using `iframe` element.

Handling iFrame

There are two ways to handle iframe.

1. frameLocator
2. frame object
3. contentFrame

```
<html>
  <iframe class="outerFrame" name="outerFrameName">
    <iframe class="innnerFrame" name="innerFrameName">
      <button id="btn">Click me</button>
    </iframe>
  </iframe>
</html>
```

frameLocator

- auto waits for elements
- reducess flakiness
- works well with nested frames
- meta data is not present using frameLocator

```
const frame = await page.frameLocator(".innerFrame");
frame.locator("#btn").click();
```

frame Object

Using frame object we get extra methods to operate on frames such as

- frame name
- Url
- childFrame
- parentFrame
- isDetached
- page

Any frame can be selected using two methods in frame object

- by url
- by name

By URL

```
const frame = page.frame({ url: "https://frame-url.com" });
frame.locator("#btn").click();
```

By Name

```
const frame = page.frame("innerFrameName");
frame.locator("#btn").click();
```

Different Methods Present In Frame

```
const frame = page.frame("frameName");
frame.url();
frame.name();
frame.childFrame();
frame.parentFrame();
frame.isDetached();
```

All Frames

To get all frames we have method `page.frames()`. This will get all the frames inside main frame or we can also provide `name` or `url` inside `frames()` method.

```
const frames = page.frames();
```

Frame traversing

frame traversing doen by

- frame.paretnFrame()
- frame.page()

where `frame.pratentFrame()` it traverse immediate frame one level up and `frame.page()` it switch all context to main page instead of one step at a time.

> *frameLocator is most preffered way to interact with iframe in playwright. But they dont provide meta inforamtion like name, url, isDetached for that we use frame object.*

iFrame in selenium

```
// 1. By index (order in the DOM, 0-based)
driver.switchTo().frame(0);

// 2. By name or id attribute
driver.switchTo().frame("frameNameOrId");

// 3. By WebElement (most robust — locate the iframe like any element)
WebElement frame = driver.findElement(By.cssSelector("iframe#myFrame"));
driver.switchTo().frame(frame);
```

The WebElement approach is preferred because index is fragile (breaks if frame order changes) and not every frame has a name/id.

- `defaultContent()` — jumps all the way back to the top-level page.
- `parentFrame()` — moves up just one level (useful for nested frames).

Waiting for a frame (recommended)

Frames may load asynchronously, so combine with `WebDriverWait`:

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.frameToBeAvailableAndSwitchToIt(By.id("myFrame")));
```

Common gotchas

- `NoSuchElementException` when an element "should" be there → you're usually in the wrong frame context. Check whether you forgot to switchTo().frame(...) or forgot to switchTo().defaultContent().
- After switching into a frame, all subsequent findElement calls are scoped to that frame until you switch out.

Comparison Tables

Playwright: frameLocator vs frame Object

Aspect	frameLocator	frame Object
Auto-waits for elements	Yes	No
Flakiness	Low (reduces flakiness)	Higher
Nested frames	Works well	Supported but manual
Frame metadata (name, url, etc.)	Not available	Available
Selecting a frame	By selector	By name or url

Aspect	frameLocator	frame Object
Extra methods (childFrame, parentFrame, isDetached, page)	No	Yes
Best for	Simple element interaction	Inspecting/traversing frames

Selenium vs Playwright iFrame Handling

Aspect	Selenium (Java)	Playwright
Model	Stateful — switch context in/out	Stateless — chain straight in
Switch into frame	`driver.switchTo().frame(.)`	`page.frameLocator(.)` / `page.frame(.)`
Switch back out	Required (`defaultContent()` / `parentFrame()`)	Not needed
Auto-wait	No (need `WebDriverWait`)	Yes (built-in)
Select frame by	Index, name/id, WebElement	Selector, name, url
Scope after switching	All lookups scoped to frame until switch out	Only chained locator is scoped
Common pitfall	Wrong frame context / forgot to switch out	Few — no context to manage

Playwright Methods — Quick Revision

Method	What it does
`page.frameLocator(selector)`	Returns a frame locator (auto-waits) to act on elements inside the frame
`page.frame(name)`	Get a frame object by its name
`page.frame({ url })`	Get a frame object by its url
`page.frames()`	Returns array of all frames on the page
`frame.locator(selector)`	Locate an element inside the frame
`frame.url()`	Get the frame's url
`frame.name()`	Get the frame's name
`frame.childFrame()`	Get the child (nested) frame
`frame.parentFrame()`	Move one level up to the parent frame

Method	What it does
`frame.isDetached()`	Check if the frame is detached from DOM
`frame.page()`	Jump straight back to the main page context

Playwright → Selenium (Java) Method Mapping

Selenium has no first-class frame object, so metadata is read via `getAttribute(.)` or the JavaScript executor.

Playwright	Selenium (Java) equivalent
`frame.url()`	`frameElement.getAttribute("src")` or JS `document.URL`
`frame.name()`	`frameElement.getAttribute("name")` or JS `window.name`
`frame.parentFrame()`	`driver.switchTo().parentFrame()`
`frame.page()`	`driver.switchTo().defaultContent()`
`frame.childFrame()`	switch into the nested frame (`switchTo().frame(.)`)
`frame.isDetached()`	no direct equivalent (catch `StaleElementReferenceException`)
`page.frames()`	`driver.findElements(By.tagName("iframe"))` (returns elements, not frame objects)
`page.frameLocator(sel).locator(.)`	`switchTo().frame(.)` then `findElement(.)`

Dialog

There are three types of dialog in playwright,

- alert
- confirm
- prompt

By default playwright **auto dismisses** every dialog. To actually handle them we need to listen for the `dialog` event using an `diloag handler`.

```
page.on("dialog", async (d)=> {
  await d.accept();
  await d.dismiss();
  d.type();
  d.message();
  d.defaultValues();
});
```

> *Important: the `dialog handler` is always written **before** the action that triggers the alert, so it does not miss the dialog.*

> *Important: inside the handler you **must** call `d.accept()` or `d.dismiss()`. If you add a handler but never accept/dismiss, the dialog stays open and the test hangs.*

Events used to listen dialog

- `page.on()` ⇒ listens for the event continuously once activated
- `page.once()` ⇒ listens only once for the defined event
- `page.waitForEvent("dialog")` ⇒ waits for and returns the dialog (used when you want to listen inline)

Different methods in dialog

- `d.type()` ⇒ gives type of dialog (`alert` / `confirm` / `prompt`)
- `d.message()` ⇒ gives text present on the dialog
- `d.defaultValue()` ⇒ gives the default value of a `prompt` dialog
- `d.accept()` ⇒ clicks OK on the dialog
- `d.dismiss()` ⇒ dismisses (clicks Cancel) on the dialog
- `d.accept("john")` ⇒ sends text into a `prompt` input then accepts

simple alert

Simple alert with only an `OK` button. Used to show a warning message.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)=> {
  expect(d.type()).toContain("alert");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  //   await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

confirm alert

Accept or reject type alert with `OK` and `Cancel` buttons.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)=> {
  expect(d.type()).toContain("confirm");
  expect(d.message()).toContain("alert message text");

  await d.accept();
  //   await d.dismiss();
});

// click on alert button
await page.locator("#alert-btn").click();
```

prompt alert

Accept or reject type alert where the user can also send a `custom message` into an input.

```
// starts listening for 'dialog' event
page.on("dialog", async (d)=> {
  expect(d.type()).toContain("prompt");
  expect(d.message()).toContain("alert message text");

  await d.accept("Kundalik"); // text passed into the prompt input
});

// click on alert button
await page.locator("#alert-btn").click();
```

Dialog Handler

Using the dialog handler we can interact with alerts. We can inspect the dialog `type` and `message`, then either `accept` or `dismiss` them, and for a `prompt` alert we can provide an input value.

Interview Questions

Q - Why do we add the dialog handler before clicking the alert button?

Ans:-

- The alert appears immediately when we click the alert button.
- JS is async in nature, so the alert can fire before a handler placed after the click is ready.
- To handle the alert we must already be listening on the `dialog` event; if the handler is placed after the click, we may lose the dialog.
- So the dialog handler is always placed before clicking the alert.

Q - What is the difference between page.on() and page.once()?

Ans:-

- `page.on()` ⇒ once activated, listens for **every** dialog
- `page.once()` ⇒ listens **only once** after it is activated

Q - What happens if we don't handle a dialog?

Ans:-

- Playwright auto dismisses it by default.
- But if a handler is registered and we never call `accept()` / `dismiss()`, the test will hang/timeout.

Alerts in Selenium

In Selenium Java an alert is handled using the `Alert` interface. We first switch to the alert, then act on it.

```
// switch to the alert
Alert alert = driver.switchTo().alert();

alert.accept();              // click OK
alert.dismiss();             // click Cancel
alert.getText();             // read the alert message
alert.sendKeys("kundalik");  // type into a prompt alert
```

Playwright vs Selenium — Alert Handling

Aspect	Playwright	Selenium (Java)
Model	Event driven (listen for `dialog`)	Switch to alert (`switchTo().alert()`)
When to set up	Handler **before** the trigger click	Switch **after** the alert appears
Default behaviour	Auto dismisses unless handled	Stays open until handled (else error)
Get type	`d.type()`	No direct method
Read message	`d.message()`	`alert.getText()`
Accept / OK	`d.accept()`	`alert.accept()`
Dismiss / Cancel	`d.dismiss()`	`alert.dismiss()`
Prompt input	`d.accept("text")`	`alert.sendKeys("text")` then `accept()`
Not handled	Auto dismissed	`UnhandledAlertException`

Playwright → Selenium (Java) Method Mapping

Playwright	Selenium (Java) equivalent
`page.on("dialog", handler)`	`driver.switchTo().alert()`
`d.message()`	`alert.getText()`
`d.accept()`	`alert.accept()`
`d.dismiss()`	`alert.dismiss()`

Playwright	Selenium (Java) equivalent
`d.accept("text")`	`alert.sendKeys("text")` then `alert.accept()`
`d.type()`	no direct equivalent
`d.defaultValue()`	no direct equivalent

Reporting in Playwright

Playwright has different built in reports

- List ⇒ simple list in command line
- Dot ⇒ ..F
- html ⇒ html report is created
- line ⇒ 4 passed
- json
- junit
- github actions
- custome reports
  - allure
  - monocart
  - tesults
  - current JS
  - serenity JS

Report configuration

For config any report we need to add `report type` inside `playwrigh.config.js` file as shown below. Or we can run any report by passing argument in `CLI` as `npx playwright test--report=list`

```
// playwright.config.js
reporter: "list";

// for multiple report to generate
reporter: [["list"], ["dot"], ["html"]];
```

Allure Report

Allure report can be generated using allure npm package.

Steps to create allure report

- Install `pnpm add -D @playwright/test allure-playwright`

- add this `reporter:[['allure-playwright', {resultsDir:'my-allure-results'}]]` to `playwright.config.js`
- Install `CLI` command to run allure report as `pnpm install -g allure-commandline--save-dev`
- Run test and then run this command `allure generate my-allure-results -o allure-report--clean`
- To open allure report run this `allure open allure-report`

Mouse, Keyboard Actions in Playwright

In Playwright, mouse and keyboard actions are available directly on the `locator` (high level) or on `page.mouse` / `page.keyboard` (low level). The common actions are:

- hover
- hover and click
- right click
- double click
- drag and drop
- click and hold
- keyboard actions

> *Note: Unlike Selenium, Playwright does **not** have an `Actions` builder class. Each action is its own auto-waiting method on the locator, so you don't need a `perform()` call at the end.*

Mouse Hover

Used to hover over a dropdown, tooltip, or help text so it becomes visible.

```
await page.locator("#help").hover();
```

Right Click

Right click on any element using the `button` option.

```
await page.locator("#rightClickBtn").click({ button: "right" });
```

The `button` option accepts:

- `left` (default)
- `right`
- `middle`

Double Click

To perform a double click we use the `dblclick()` method.

```
await page.locator("#db-click").dblclick();
```

Drag and Drop

Playwright provides a built-in method `source.dragTo(target)`. There is also a low-level manual approach using `page.mouse`.

```
const source = page.locator("#src");
const target = page.locator("#trg");

// Approach 01 - manual (low level)
await source.hover();
await page.mouse.down(); // press mouse on source

await target.hover();
await page.mouse.up(); // release on target

// Approach 02 - built in (preferred)
await source.dragTo(target);
```

Keyboard actions

Playwright handles keys with `page.keyboard.press()`. Shortcuts are written with `+`.

```
// Shortcut / multiple keys together
await page.keyboard.press("Control+A");
await page.keyboard.press("Shift+A");

// Single key
await page.keyboard.press("a");
await page.keyboard.press("Enter");

// Type a whole string character by character (fires individual key events)
// `type()` is deprecated, use `pressSequentially()` instead
await page.locator("#search").pressSequentially("kundalik", { delay: 100 });
```

> *`press()` is one key/shortcut at a time. `pressSequentially()` types a full string character by character and fires real key events (the `delay` option, in ms, adds a pause between each key — useful for inputs that react on every keystroke). It replaces the deprecated `type()`. To fill a field quickly without per-key events, prefer `locator.fill("text")`.*

Actions in Selenium Java

In Selenium all these actions are performed using the `Actions` class.

```
Actions act = new Actions(driver);
```

- The `Actions` class supports **method chaining**.
- It follows the **builder pattern**.
- We can chain multiple actions together, but to actually run them we must call `perform()` at the end.
- Chaining makes complex operations readable and efficient.

> *Note: each block below is a **standalone example** of one gesture. If you ever run several gestures back-to-back on the **same** `act` instance, remember Selenium accumulates queued actions and `perform()` does not clear them — so use a fresh `Actions` instance per gesture in real tests. `Duration.ofSeconds(.)` needs `import java.time.Duration;`.*

```
WebDriver driver = new ChromeDriver();
Actions act = new Actions(driver);

WebElement element = driver.findElement(By.id("help"));

// Mouse hover
act.moveToElement(element)
   .perform();

// Hover and click
act.moveToElement(element)
   .click()
   .perform();

// Right click
act.contextClick(element)
   .perform();

// Double click
act.doubleClick(element)
   .perform();

// Click and hold
act.clickAndHold(element)
   .perform();

// Click, hold, then release
act.clickAndHold(element)
   .pause(Duration.ofSeconds(2))
   .release()
   .perform();
```

Drag and Drop

To drag and drop we use the `dragAndDrop(source, target)` method with the source and target elements.

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.dragAndDrop(source, target)
   .perform();
```

Sometimes `dragAndDrop(src, trg)` won't work because of custom JS-based drag scripts that Selenium's method cannot capture. In that case use the manual click-hold-move-release approach:

```
WebElement source = driver.findElement(By.id("src"));
WebElement target = driver.findElement(By.id("trg"));

act.clickAndHold(source)
   .moveToElement(target)
   .release()
   .perform();
```

Move using co-ordinates

To drag by a pixel offset instead of to a target element:

```
act.dragAndDropBy(source, xOffset, yOffset)
   .perform();
```

Here `xOffset` and `yOffset` are the distances to move in `px`.

KeyPress in Selenium

The `Keys` enum is used to perform keyboard operations in Selenium.

```
import org.openqa.selenium.Keys;

WebElement searchLoc = driver.findElement(By.name("q"));

act.sendKeys(searchLoc, "kundalik")
   .sendKeys(Keys.ENTER)
   .perform();
```

- `keyDown()` ⇒ simulates pressing and holding a key.
- `keyUp()` ⇒ simulates releasing the held key.

Performing Ctrl + A

```
act.keyDown(Keys.CONTROL)   // press & hold Ctrl
   .sendKeys("a")           // press A
   .keyUp(Keys.CONTROL)     // release Ctrl
   .perform();
```

Interview Questions

Q - Why do we use keyDown before keyUp?

Ans:-

- `keyDown()` presses and holds the key.
- `keyUp()` releases the held key.
- This mimics how a real user interacts: they press and hold the first key, press the second key, then release. Without `keyUp`, the key stays logically held down.

Q - Why does Selenium need perform() but Playwright does not?

Ans:-

- Selenium's `Actions` uses the builder pattern — it only builds a queue of actions; `perform()` actually executes them.
- Playwright actions are individual auto-waiting methods that execute immediately when awaited, so no `perform()` is needed.

Q - Difference between dragTo / dragAndDrop and the manual approach?

Ans:-

- The built-in method is one line and works for standard HTML drag.
- The manual approach (hover/down/move/up) is used when the built-in method fails on custom JS drag-and-drop implementations.

Q - hover() use cases?

Ans:-

- Revealing hidden menus, dropdowns, tooltips, or help text that only appear on hover.

Action Methods Comparison

Action	Playwright	Selenium (Java)
Hover	`locator.hover()`	`act.moveToElement(el).perform()`
Hover + click	`locator.hover()` then `locator.click()`	`act.moveToElement(el).click().perform()`
Right click	`locator.click({ button: "right" })`	`act.contextClick(el).perform()`
Double click	`locator.dblclick()`	`act.doubleClick(el).perform()`
Click and hold	`page.mouse.down()`	`act.clickAndHold(el).perform()`
Release	`page.mouse.up()`	`act.release().perform()`
Drag and drop	`source.dragTo(target)`	`act.dragAndDrop(src, trg).perform()`

Action	Playwright	Selenium (Java)
Move by offset	`page.mouse.move(x, y)`	`act.dragAndDropBy(src, x, y).perform()`
Press key / shortcut	`page.keyboard.press("Control+A")`	`act.keyDown(Keys.CONTROL).sendKeys("a").keyUp(Keys.CONTROL).perform()`
Type string	`locator.pressSequentially("text", { delay: 100 })`	`act.sendKeys("text").perform()`
Needs perform()	No (auto executes)	Yes (builder pattern)

Keys Comparison

Selenium	Playwright
`Keys.ENTER`	`"Enter"`
`Keys.CONTROL`	`"Control"`
`Keys.SHIFT`	`"Shift"`
`Keys.TAB`	`"Tab"`
`Keys.ESCAPE`	`"Escape"`
`Keys.ARROW_DOWN`	`"ArrowDown"`
